Квантово-вдохновлённые подходы

Выполнила команда: Страдальцы (АНИСА)

Мягкова А.А., Воробьёва А.А., Синий Н.А., Потапова С.А.

```python
import numpy as np
import scipy.sparse as sparse
import time
```

1 – векторные и матричные операции

2 – разреженные (sparse) матрицы

3 – измерение времени выполнения и контроля за общим лимитом

```python
def solve(matrix: np.ndarray) -> np.ndarray:
    n = matrix.shape[0]
    start_time = time.time()
    max_time = 9.5
    if np.count_nonzero(matrix) < 0.1 * n * n:
        Q_sparse = sparse.csr_matrix(matrix)
        is_sparse = True
    else:
        Q_sparse = None
        is_sparse = False
```

$$H(x) = x^T Q x$$

(10)-(15) оптимизация для разреженных матриц. Если матрица разреженная, то конвертируется в формат Compressed Sparse Row

Объяснение: оптимизация производительности – использование разреженного формата ускоряет вычисление для больших и слабосвязанных графов

```python
    y = np.random.uniform( low: 0,  high: 1, n)

    best_x = (y > 0.5).astype(np.int8)
    if is_sparse:
        best_energy = best_x @ Q_sparse @ best_x
    else:
        best_energy = best_x @ matrix @ best_x

    T = T0
    step_count = 0
```

(42) Получает нормальный бинарный вектор **x** путем округления **y** к нулю или единице

$$E = x^T Q x$$

```python
while time.time() - start_time < max_time and T > T_min:
    for _ in range(max_inner):

        if is_sparse:
            v = Q_sparse.dot(y)
        else:
            v = matrix.dot(y)

        Phi = 2 * v - diag_Q * (2 * y - 1)

        y_new = 1 / (1 + np.exp(Phi / T))

        y = (1 - lam) * y + lam * y_new
```

$$\Phi_i = 2\sum_j Q_{ij}y_j - Q_{ii}(2y_i - 1)$$

```python
        step_count += 1
        if step_count % check_interval == 0:
            x_current = (y > 0.5).astype(np.int8)
            if is_sparse:
                energy = x_current @ Q_sparse @ x_current
            else:
                energy = x_current @ matrix @ x_current

            if energy < best_energy:
                best_energy = energy
                best_x = x_current.copy()

        # Охлаждение
        T *= alpha
```

(67) – получение текущего бинарного вектора x из y
(68)-(71) вычисление фактической энергии, если энергия лучше (меньше), то
(74) – обновление лучшей энергии и (75) сохраняется новый лучший бинарный код

```python
    x_final = (y > 0.5).astype(np.int8)
    if is_sparse:
        final_energy = x_final @ Q_sparse @ x_final
    else:
        final_energy = x_final @ matrix @ x_final

    if final_energy < best_energy:
        return x_final
    else:
        return best_x
```

Перспективные идеи модификации и оптимизации

Вместо геометрической прогрессии $T_{k+1} = \alpha \mathrm{T}_k$

Понижение температуры по формуле $T_k = \frac{T_0}{k^p}$

может дать лучшие теоретические гарантии сходимости, но требует более тонкой настройки. Вместо полностью случайного y, можно применить быстрый жадный алгоритм (например, основываясь на степенях вершин графа) для создания начального бинарного решения x. Для ускорения вычислений, которые не могут быть полностью векторизованы (хотя в нашем коде большинство векторизовано) или для ускорения циклов, можно использовать компилятор numba (с помощью декоратора @numba.jit). Это может дать выигрыш в скорости, но не всегда совместимо с scipy.sparse.

Задача 2

```
1  import sys
2  from collections import deque
3  from typing import Dict, List, Tuple, Set, Optional
```

1 – импорт системного модуля
2 – нужно для эффективного поиска в ширину (BFS)
3 – импорт аннотаций для повышения читаемости и надёжности кода

```python
Edge = Tuple[int, int]
Graph = Dict[int, List[int]]
WavelengthUsage = Dict[int, Set[Edge]]
```

(5): ребро – это кортеж из двух целых чисел (начальная и конечная вершина).

(6): граф – это словарь, где ключ (int) — вершина, а значение (List[int]) — список вершин, в которые можно попасть из неё.

(7): использование длин волн – это словарь, где ключ (int) — номер длины волны, а значение (Set[Edge]) — множество рёбер, которые заняты на этой длине волны.

```python
def read_graph(graph_file) -> Graph:  1 usage
    with open(graph_file, 'r', encoding='utf-8') as f:
        lines = [line.strip() for line in f if line.strip()]

    if not lines:
        return {}

    try:
        n, m = map(int, lines[0].split())
    except ValueError:
        return {}

    edges = [tuple(map(int, line.split())) for line in lines[1:]]
    graph = {i: [] for i in range(n)}
    for u, v in edges:
        if u in graph and v in graph:
            graph[u].append(v)
    return graph
```

(13)-(14) обработка случая пустого файла

```python
def read_requests(data_file) -> Tuple[int, List[Tuple[int, int]]]:  1 usage
    with open(data_file, 'r', encoding='utf-8') as f:
        lines = [line.strip() for line in f if line.strip()]

    if len(lines) < 2:
        return 0, []

    try:
        num_wavelengths = int(lines[0])
        num_requests = int(lines[1])
    except ValueError:
        return 0, []

    requests = []
    for line in lines[2:2 + num_requests]:
        try:
            requests.append(tuple(map(int, line.split())))
        except ValueError:
            continue

    return num_wavelengths, requests
```

Вспомогательные функции для путей

```
def path_to_edges(path: List[int]) -> List[Edge]:  1 usage
    return list(zip(path, path[1:]))
```

Функция для преобразования пути (списка вершин) в список рёбер.

Использует zip для создания пар последовательных вершин, представляющих рёбра пути.

```python
def find_path(graph: Graph, start: int, end: int,   1 usage
              wavelength_usage: WavelengthUsage, wavelength: Optional[int] = None,
              exclude_edges: Optional[Set[Edge]] = None) -> Optional[List[int]]:
    if start == end:
        return [start]

    queue = deque([(start, [start])])
    visited = {start}

    occupied_edges = wavelength_usage.get(wavelength, set()) if wavelength is not None else set()
    excluded = exclude_edges if exclude_edges is not None else set()

    while queue:
        node, path = queue.popleft()

        for neigh in graph.get(node, []):
            edge = (node, neigh)

            if edge not in excluded and edge not in occupied_edges:
                if neigh == end:
                    return path + [neigh]

                if neigh not in visited:
                    visited.add(neigh)
                    queue.append((neigh, path + [neigh]))

    return None
```

```python
def find_available_wavelength(graph: Graph, start: int, end: int, num_wavelengths: int,  1 usage
                              wavelength_usage: WavelengthUsage,
                              exclude_edges: Optional[Set[Edge]] = None) -> Tuple[Optional[List[int]],Optional[int]]:
    best_path = None
    best_wavelength = None

    for wavelength in range(num_wavelengths):
        path = find_path(graph, start, end, wavelength_usage, wavelength=wavelength, exclude_edges=exclude_edges)

        if path is not None:
            best_path = path
            best_wavelength = wavelength
            break

    return best_path, best_wavelength

```

```python
def solve(graph_file: str, data_file: str) -> Dict[Tuple[int, int], Dict[str, Dict[str, any]]]:
    graph = read_graph(graph_file)
    num_wavelengths, requests = read_requests(data_file)

    if num_wavelengths == 0:
        return {}

    wavelength_usage: WavelengthUsage = {w: set() for w in range(num_wavelengths)}

    mapping: Dict[Tuple[int, int], Dict[str, Dict[str, any]]] = {}

    default_wavelength = 0

    for src, dst in requests:
```

1) Главная функция, реализующая логику маршрутизации.
2) Чтение входных данных.
3) Защита от нулевого числа волн.
4) Инициализация словаря занятости длин волн.
5) Установка длины волны по умолчанию (используется, если путь не найден).
6) Жадный цикл по всем запросам в порядке их чтения.

```python
# 1. Поиск ОСНОВНОГО пути
primary_path, primary_wavelength = find_available_wavelength(
    graph, src, dst, num_wavelengths, wavelength_usage
)

primary_info = {
    'wavelength': primary_wavelength if primary_wavelength is not None else default_wavelength,
    'path': primary_path if primary_path is not None else []}

primary_edges = set(path_to_edges(primary_path)) if primary_path else set()

backup_path = []
backup_wavelength = None
```

1) Формирование структуры данных для основного пути.
2) Получение множества рёбер, составляющих основной путь.
3) Инициализация для резервного пути

```python
        # 2. Поиск РЕЗЕРВНОГО пути
    if primary_path:
        # Ищем резервный путь, не пересекающийся с основным
        backup_path, backup_wavelength = find_available_wavelength(
            graph, src, dst, num_wavelengths, wavelength_usage, exclude_edges=primary_edges
        )

    backup_info = {
        'wavelength': backup_wavelength if backup_wavelength is not None else default_wavelength,
        'path': backup_path if backup_path is not None else []
    }
```

```python
        # 3. Регистрация занятости ребер
        if primary_path and primary_wavelength is not None:
            if 0 <= primary_wavelength < num_wavelengths:
                wavelength_usage[primary_wavelength].update(primary_edges)

        if backup_path and backup_wavelength is not None:
            if 0 <= backup_wavelength < num_wavelengths:
                backup_edges = set(path_to_edges(backup_path))
                wavelength_usage[backup_wavelength].update(backup_edges)
```

```python
        # 4. Формирование результата
        mapping[(src, dst)] = {
            'primary': primary_info,
            'backup': backup_info
        }


    return mapping
```

Перспективные идеи модификации и оптимизации

Вместо обработки в порядке поступления, можно применить следующие эвристики:

- Сортировка по сложности: Обрабатывать сначала самые сложные запросы. Сложность можно определить по:
- Длине кратчайшего пути: Запросы с более длинным кратчайшим путём могут быть труднее выполнить.
- Числу доступных путей: Запросы, имеющие мало альтернативных путей, должны быть обработаны первыми.
- "Стоимости" запроса: Запросы, использующие рёбра, которые входят в состав кратчайших путей для многих других запросов, следует обрабатывать раньше, чтобы обеспечить их выполнение.

Модификация: Вместо простого BFS для поиска кратчайшего пути, можно использовать алгоритм K-кратчайших путей (например, алгоритм Йена).

Это позволило бы сгенерировать несколько альтернативных пар (основной/резервный) и выбрать ту, которая минимально влияет на доступность рёбер для остальных запросов (например, выбирая пару, использующую наименее популярные/критичные рёбра).